

Modern Operating Systems — Chapter 5 Input/Output Exercises and Answers

Andrew S. Tanenbaum and Herbert Bos

Contents

Chapter 5: Input/Output	2
Exercise 1	2
Exercise 2	2
Exercise 3	3
Exercise 4	3
Exercise 5	4
Exercise 6	4
Exercise 7	5
Exercise 8	6
Exercise 9	6
Exercise 10	7
Exercise 11	7
Exercise 12	8
Exercise 13	8
Exercise 14	9
Exercise 15	9

Exercise 16	10
Exercise 17	10
Exercise 18	11
Exercise 19	11
Exercise 20	12
Exercise 21	12
Exercise 22	13
Exercise 23	13
Exercise 24	14
Exercise 25	14
Exercise 26	14
Exercise 27	15
Exercise 28	15
Exercise 29	16
Exercise 30	16
Exercise 31	17
Exercise 32	17
Exercise 33	18
Exercise 34	18
Exercise 35	19
Exercise 36	19

Exercise 37	20
Exercise 38	20
Exercise 39	21
Exercise 40	21
Exercise 41	22
Exercise 42	22
Exercise 43	23
Exercise 44	23
Exercise 45	24
Exercise 46	24
Exercise 47	25
Exercise 48	25
Exercise 49	25
Exercise 50	26
Exercise 51	26
Exercise 52	27
Exercise 53	27
Exercise 54	28
Exercise 55	28
Exercise 56	29
Exercise 57	30

Chapter 5: Input/Output

Source note. Questions were extracted from the Chapter 5 problem section of *Modern Operating Systems*, 4th edition. Answers 1–32 were matched to the same-numbered Chapter 5 answer-manual items. Because the answer manual contains an extra item numbered 33 that does not match textbook Exercise 33, textbook Exercises 33–54 were matched by content to answer-manual items 34–55. Exercises 55–57 had no matching answer in that answer section, so their answers are clearly labeled as prepared technical answers rather than answer-manual text. Additional math explanations are separately labeled where added.

Exercise 1

Question

Advances in chip technology have made it possible to put an entire controller, including all the bus access logic, on an inexpensive chip. How does that affect the model of Fig. 1-6?

Answer

In the figure, we see controllers and devices as separate units. The reason is to allow a controller to handle multiple devices, and thus eliminate the need for having a controller per device. If controllers become almost free, then it will be simpler just to build the controller into the device itself. This design will also allow multiple transfers in parallel and thus give better performance.

Source: Answer manual.

Exercise 2

Question

Given the speeds listed in Fig. 5-1, is it possible to scan documents from a scanner and transmit them over an 802.11g network at full speed? Defend your answer.

Answer

Easy. The scanner puts out 400 KB/sec maximum. The wireless network runs at 6.75 MB/sec, so there is no problem at all.

Source: Answer manual.

Exercise 3

Question

Figure 5-3(b) shows one way of having memory-mapped I/O even in the presence of separate buses for memory and I/O devices, namely, to first try the memory bus and if that fails try the I/O bus. A clever computer science student has thought of an improvement on this idea: try both in parallel, to speed up the process of accessing I/O devices. What do you think of this idea?

Answer

It is not a good idea. The memory bus is surely faster than the I/O bus, otherwise why bother with it? Consider what happens with a normal memory request. The memory bus finishes first, but the I/O bus is still busy. If the CPU waits until the I/O bus finishes, it has reduced memory performance to that of the I/O bus. If it just tries the memory bus for the second reference, it will fail if this one is an I/O device reference. If there were some way to instantaneously abort the previous I/O bus reference to try the second one, the improvement might work, but there is never such an option. All in all, it is a bad idea.

Source: Answer manual.

Exercise 4

Question

Explain the tradeoffs between precise and imprecise interrupts on a superscalar machine.

Answer

An advantage of precise interrupts is simplicity of code in the operating system since the machine state is well defined. On the other hand, in imprecise interrupts, OS writers have to figure out what instructions have been partially executed and up to what point. However, precise interrupts increase complexity of chip design and chip area, which may result in slower CPU.

Source: Answer manual.

Exercise 5

Question

A DMA controller has five channels. The controller is capable of requesting a 32-bit word every 40 nsec. A response takes equally long. How fast does the bus have to be to avoid being a bottleneck?

Answer

Each bus transaction has a request and a response, each taking 50 nsec, or 100 nsec per bus transaction. This gives 10 million bus transactions/sec. If each one is good for 4 bytes, the bus has to handle 40 MB/sec. The fact that these transactions may be sprayed over five I/O devices in round-robin fashion is irrelevant. A bus transaction takes 100 nsec, regardless of whether consecutive requests are to the same device or different devices, so the number of channels the DMA controller has does not matter. The bus does not know or care.

Source: Answer manual.

Math explanation. Each DMA bus transaction transfers one 32-bit word, i.e., 4 bytes. The answer manual uses 100 nsec per complete transaction, so the transaction rate is $1/(100 \text{ nsec}) = 10^7$ transactions/sec. Multiplying by 4 bytes gives 4×10^7 bytes/sec, or 40 MB/sec.

Source: Prepared from textbook concepts.

Exercise 6

Question

Suppose that a system uses DMA for data transfer from disk controller to main memory. Further assume that it takes t_1 nsec on average to acquire the bus and t_2 nsec to transfer one word over the bus ($t_1 \gg t_2$). After the CPU has programmed the DMA controller, how long will it take to transfer 1000 words from the disk controller to main memory, if

- (a) word-at-a-time mode is used,
- (b) burst mode is used? Assume that commanding the disk controller requires acquiring the bus to send one word and acknowledging a transfer also requires acquiring the bus to send one word.

Answer

(a) Word-at-a-time mode: $1000 \times [(t_1 + t_2) + (t_1 + t_2) + (t_1 + t_2)]$ Where the first term is for acquiring the bus and sending the command to the disk controller, the second term is for transferring the word, and the third term is for the acknowledgement. All in all, a total of $3000 \times (t_1 + t_2)$ nsec.

(b) Burst mode: $(t_1 + t_2) + t_1 + 1000 \text{ times } t_2 + (t_1 + t_2)$ where the first term is for acquiring the bus and sending the command to the disk controller, the second term is for the disk controller to acquire the bus, the third term is for the burst transfer, and the fourth term is for acquiring the bus and doing the acknowledgement. All in all, a total of $3t_1 + 1002t_2$.

Source: Answer manual.

Math explanation. In word-at-a-time mode, each of 1000 words needs three bus uses: command, data transfer, and acknowledgement. Each bus use costs $t_1 + t_2$, so the total is $1000 \times 3(t_1 + t_2) = 3000(t_1 + t_2)$. In burst mode, the command costs $t_1 + t_2$, the controller gets the bus once for t_1 , the 1000 data words cost $1000t_2$, and the acknowledgement costs $t_1 + t_2$. Adding them gives $3t_1 + 1002t_2$.

Source: Prepared from textbook concepts.

Exercise 7

Question

One mode that some DMA controllers use is to have the device controller send the word to the DMA controller, which then issues a second bus request to write to memory. How can this mode be used to perform memory to memory copy? Discuss any advantage or disadvantage of using this method instead of using the CPU to perform memory to memory copy.

Answer

Memory to memory copy can be performed by first issuing a read command that will transfer the word from memory to DMA controller and then issuing a write to memory to transfer the word from the DMA controller to a different address in memory. This method has the advantage that the CPU can do other useful work in parallel. The disadvantage is that this memory to memory copy is likely to be slow since DMA controller is much slower than CPU and the data transfer takes place over system bus as opposed to the dedicated CPU-memory bus.

Source: Answer manual.

Exercise 8

Question

Suppose that a computer can read or write a memory word in 5 nsec. Also suppose that when an interrupt occurs, all 32 CPU registers, plus the program counter and PSW are pushed onto the stack. What is the maximum number of interrupts per second this machine can process?

Answer

An interrupt requires pushing 34 words onto the stack. Returning from the interrupt requires fetching 34 words from the stack. This overhead alone is 340 nsec. Thus the maximum number of interrupts per second is no more than about 2.94 million, assuming no work for each interrupt.

Source: Answer manual.

Math explanation. The interrupt saves 32 registers plus PC plus PSW, or 34 words. Returning restores 34 words. That is 68 memory-word operations. At 5 nsec each, the save/restore overhead is $68 \times 5 = 340$ nsec. The theoretical upper bound is $1/340 \text{ nsec} \approx 2.94 \times 10^6 \text{ interrupts/sec}$, before doing any useful interrupt work.

Source: Prepared from textbook concepts.

Exercise 9

Question

CPU architects know that operating system writers hate imprecise interrupts. One way to please the OS folks is for the CPU to stop issuing new instructions when an interrupt is signaled, but allow all the instructions currently being executed to finish, then force the interrupt. Does this approach have any disadvantages? Explain your answer.

Answer

The execution rate of a modern CPU is determined by the number of instructions that finish per second and has little to do with how long an instruction takes. If a CPU can finish 1 billion instructions/sec it is a 1000 MIPS machine, even if an instruction takes 30 nsec. Thus there is generally little attempt to make instructions finish quickly. Holding the interrupt until the last instruction currently executing finishes may increase the latency of interrupts appreciably. Furthermore, some administration is required to get this right.

Source: Answer manual.

Exercise 10

Question

In Fig. 5-9(b), the interrupt is not acknowledged until after the next character has been output to the printer. Could it have equally well been acknowledged right at the start of the interrupt service procedure? If so, give one reason for doing it at the end, as in the text. If not, why not?

Answer

It could have been done at the start. A reason for doing it at the end is that the code of the interrupt service procedure is very short. By first outputting another character and then acknowledging the interrupt, if another interrupt happens immediately, the printer will be working during the interrupt, making it print slightly faster. A disadvantage of this approach is slightly longer dead time when other interrupts may be disabled.

Source: Answer manual.

Exercise 11

Question

A computer has a three-stage pipeline as shown in Fig. 1-7(a). On each clock cycle, one new instruction is fetched from memory at the address pointed to by the PC and put into the pipeline and the PC advanced. Each instruction occupies exactly one memory word. The instructions already in the pipeline are each advanced one stage. When an interrupt occurs, the current PC is pushed onto the stack, and the PC is set to the address of the interrupt handler. Then the pipeline is shifted right one stage and the first instruction of the interrupt handler is fetched into the pipeline. Does this machine have precise interrupts? Defend your answer.

Answer

Yes. The stacked PC points to the first instruction not fetched. All instructions before that have been executed and the instruction pointed to and its successors have not been executed. This is the condition for precise interrupts. Precise interrupts are not hard to achieve on machine with a single pipeline. The trouble comes in when instructions are executed out of order, which is not the case here.

Source: Answer manual.

Exercise 12

Question

A typical printed page of text contains 50 lines of 80 characters each. Imagine that a certain printer can print 6 pages per minute and that the time to write a character to the printer's output register is so short it can be ignored. Does it make sense to run this printer using interrupt-driven I/O if each character printed requires an interrupt that takes $50\ \mu\text{sec}$ all-in to service?

Answer

The printer prints $50 \times 80 \times 6 = 24,000$ characters/min, which is 400 characters/sec. Each character uses $50\ \mu\text{sec}$ of CPU time for the interrupt, so collectively in each second the interrupt overhead is 20 msec. Using interrupt-driven I/O, the remaining 980 msec of time is available for other work. In other words, the interrupt overhead costs only 2% of the CPU, which will hardly affect the running program at all.

Source: Answer manual.

Math explanation. The printer produces $50 \times 80 \times 6 = 24,000$ characters/minute, or 400 characters/sec. Each character interrupt costs 50 microseconds, so CPU time per second is $400 \times 50\ \mu\text{s} = 20,000\ \mu\text{s} = 20\ \text{msec}$. Since one second has 1000 msec, this is $20/1000 = 2\%$ of the CPU.

Source: Prepared from textbook concepts.

Exercise 13

Question

Explain how an OS can facilitate installation of a new device without any need for recompiling the OS.

Answer

UNIX does it as follows. There is a table indexed by device number, with each table entry being a C struct containing pointers to the functions for opening, closing, reading, and writing and a few other things from the device. To install a new device, a new entry has to be made in this table and the pointers filled in, often to the newly loaded device driver.

Source: Answer manual.

Exercise 14

Question

In which of the four I/O software layers is each of the following done.

- (a) Computing the track, sector, and head for a disk read.
- (b) Writing commands to the device registers.
- (c) Checking to see if the user is permitted to use the device.
- (d) Converting binary integers to ASCII for printing.

Answer

- (a) Device driver.
- (b) Device driver.
- (c) Device-independent software.
- (d) User-level software.

Source: Answer manual.

Exercise 15

Question

A local area network is used as follows. The user issues a system call to write data packets to the network. The operating system then copies the data to a kernel buffer. Then it copies the data to the network controller board. When all the bytes are safely inside the controller, they are sent over the network at a rate of 10 megabits/sec. The receiving network controller stores each bit a microsecond after it is sent. When the last bit arrives, the destination CPU is interrupted, and the kernel copies the newly arrived packet to a kernel buffer to inspect it. Once it has figured out which user the packet is for, the kernel copies the data to the user space. If we assume that each interrupt and its associated processing takes 1 msec, that packets are 1024 bytes (ignore the headers), and that copying a byte takes 1 μ sec, what is the maximum rate at which one process can pump data to another? Assume that the sender is blocked until the work is finished at the receiving side and an acknowledgement comes back. For simplicity, assume that the time to get the acknowledgement back is so small it can be ignored.

Answer

A packet must be copied four times during this process, which takes 4.1 msec. There are also two interrupts, which account for 2 msec. Finally, the transmission time is 0.83 msec, for a total of 6.93 msec per 1024 bytes. The maximum data rate is thus 147,763 bytes/sec, or about 12% of the nominal 10 megabit/sec network capacity. (If we include protocol overhead, the figures get even worse.)

Source: Answer manual.

Math explanation. A 1024-byte packet is copied four times, so copying costs about $4 \times 1024\mu s = 4096\mu s \approx 4.1$ msec. Two interrupts add 2 msec. Transmission at 10 Mbit/sec for $1024 \times 8 = 8192$ bits costs about 0.819 msec. Total time is about 6.93 msec, so throughput is $1024/0.00693 \approx 147,763$ bytes/sec.

Source: Prepared from textbook concepts.

Exercise 16

Question

Why are output files for the printer normally spooled on disk before being printed?

Answer

If the printer were assigned as soon as the output appeared, a process could tie up the printer by printing a few characters and then going to sleep for a week.

Source: Answer manual.

Exercise 17

Question

How much cylinder skew is needed for a 7200-RPM disk with a track-to-track seek time of 1 msec? The disk has 200 sectors of 512 bytes each on each track.

Answer

The disk rotates at 120 RPS, so 1 rotation takes $1000/120$ msec. With 200 sectors per rotation, the sector time is $1/200$ of this number or $5/120 = 1/24$ msec. During the 1-msec seek, 24 sectors pass under the head. Thus the cylinder skew should be 24.

Source: Answer manual.

Math explanation. A 7200-RPM disk makes $7200/60 = 120$ rotations/sec, so one rotation takes $1/120$ sec = 8.33 msec. A 1 msec track-to-track seek spans $1/8.33 \approx 0.12$ of a rotation. With 200 sectors/track, the skew should be $0.12 \times 200 = 24$ sectors.

Source: Prepared from textbook concepts.

Exercise 18

Question

A disk rotates at 7200 RPM. It has 500 sectors of 512 bytes around the outer cylinder. How long does it take to read a sector?

Answer

At 7200 RPM, there are 120 rotations per second, so 1 rotation takes about 8.33 msec. Dividing this by 500 we get a sector time of about 16.67 μ sec.

Source: Answer manual.

Math explanation. One rotation takes 8.33 msec. With 500 sectors on the track, one sector time is $8.33/500 = 0.0167$ msec, or about 16.7 microseconds. This is just rotational transfer time for one sector, not seek time.

Source: Prepared from textbook concepts.

Exercise 19

Question

Calculate the maximum data rate in bytes/sec for the disk described in the previous problem.

Answer

There are 120 rotations in a second. During one of them, 500×512 bytes pass under the head. So the disk can read 256,000 bytes per rotation or 30,720,000 bytes/sec.

Source: Answer manual.

Math explanation. One full outer-cylinder rotation reads $500 \times 512 = 256,000$ bytes. At 7200 RPM, the disk makes 120 rotations/sec. Maximum outer-cylinder data rate is therefore $256,000 \times 120 = 30,720,000$ bytes/sec.

Source: Prepared from textbook concepts.

Exercise 20

Question

RAID level 3 is able to correct single-bit errors using only one parity drive. What is the point of RAID level 2? After all, it also can only correct one error and takes more drives to do so.

Answer

RAID level 2 can not only recover from crashed drives, but also from undetected transient errors. If one drive delivers a single bad bit, RAID level 2 will correct this, but RAID level 3 will not.

Source: Answer manual.

Exercise 21

Question

A RAID can fail if two or more of its drives crash within a short time interval. Suppose that the probability of one drive crashing in a given hour is p . What is the probability of a k -drive RAID failing in a given hour?

Answer

The probability of 0 failures, P_0 , is $(1 - p)^k$. The probability of 1 failure, P_1 , is $kp(1 - p)^{k-1}$. The probability of a RAID failure is then $1 - P_0 - P_1$. This is $1 - (1 - p)^k - kp(1 - p)^{k-1}$.

Source: Answer manual.

Math explanation. If drive failures are independent and each drive has probability p of failing in an hour, the probability of a specific pair failing is p^2 . There are $\binom{k}{2} = k(k - 1)/2$ pairs. For small p , the probability of two or more failures is approximately $\binom{k}{2}p^2$; the exact expression is $1 - (1 - p)^k - kp(1 - p)^{k-1}$.

Source: Prepared from textbook concepts.

Exercise 22

Question

Compare RAID level 0 through 5 with respect to read performance, write performance, space overhead, and reliability.

Answer

Read performance: RAID levels 0, 2, 3, 4, and 5 allow for parallel reads to service one read request. However, RAID level 1 further allows two read requests to simultaneously proceed. Write performance: All RAID levels provide similar write performance. Space overhead: There is no space overhead in level 0 and 100% overhead in level 1. With 32-bit data word and six parity drives, the space overhead is about 18.75% in level 2. For a 32-bit data word, the space overhead in level 3 is about 3.13%. Finally, assuming 33 drives in levels 4 and 5, the space overhead is 3.13% in them. Reliability: There is no reliability support in level 0. All other RAID levels can survive one disk crash. In addition, in levels 3, 4 and 5, a single random bit error in a word can be detected, while in level 2, a single random bit error in a word can be detected and corrected.

Source: Answer manual.

Exercise 23

Question

How many pebibytes are there in a zebibyte?

Answer

A zebibyte is 2^{70} bytes; a pebibyte is 2^{50} . 2^{20} pebibytes fit in a zebibyte.

Source: Answer manual.

Math explanation. A zebibyte is 2^{70} bytes and a pebibyte is 2^{50} bytes. Divide: $2^{70}/2^{50} = 2^{20}$. Thus one zebibyte contains $2^{20} = 1,048,576$ pebibytes.

Source: Prepared from textbook concepts.

Exercise 24

Question

Why are optical storage devices inherently capable of higher data density than magnetic storage devices? Note: This problem requires some knowledge of high-school physics and how magnetic fields are generated.

Answer

A magnetic field is generated between two poles. Not only is it difficult to make the source of a magnetic field small, but also the field spreads rapidly, which leads to mechanical problems trying to keep the surface of a magnetic medium close to a magnetic source or sensor. A semiconductor laser generates light in a very small place, and the light can be optically manipulated to illuminate a very small spot at a relatively great distance from the source.

Source: Answer manual.

Exercise 25

Question

What are the advantages and disadvantages of optical disks versus magnetic disks?

Answer

The main advantage of optical disks is that they have much higher recording densities than magnetic disks. The main advantage of magnetic disks is that they are an order of magnitude faster than the optical disks.

Source: Answer manual.

Exercise 26

Question

If a disk controller writes the bytes it receives from the disk to memory as fast as it receives them, with no internal buffering, is interleaving conceivably useful? Discuss your answer.

Answer

Possibly. If most files are stored in logically consecutive sectors, it might be worthwhile interleaving the sectors to give programs time to process the data just received, so that

when the next request is issued, the disk would be in the right place. Whether this is worth the trouble depends strongly on the kind of programs run and how uniform their behavior is.

Source: Answer manual.

Exercise 27

Question

If a disk has double interleaving, does it also need cylinder skew in order to avoid missing data when making a track-to-track seek? Discuss your answer.

Answer

Maybe yes and maybe no. Double interleaving is effectively a cylinder skew of two sectors. If the head can make a track-to-track seek in fewer than two sector times, then no additional cylinder skew is needed. If it cannot, then additional cylinder skew is needed to avoid missing a sector after a seek.

Source: Answer manual.

Exercise 28

Question

Consider a magnetic disk consisting of 16 heads and 400 cylinders. This disk has four 100-cylinder zones with the cylinders in different zones containing 160, 200, 240, and 280 sectors, respectively. Assume that each sector contains 512 bytes, average seek time between adjacent cylinders is 1 msec, and the disk rotates at 7200 RPM. Calculate the

- (a) disk capacity,
- (b) optimal track skew, and
- (c) maximum data transfer rate.

Answer

Consider,

(a) The capacity of a zone is $\text{tracks} \times \text{cylinders} \times \text{sectors/cylinder} \times \text{bytes/sect}$. Capacity of zone 1: $16 \times 100 \times 160 \times 512 = 131072000$ bytes Capacity of zone 2: $16 \times 100 \times 200 \times 512 = 163840000$ bytes Capacity of zone 3: $16 \times 100 \times 240 \times 512 = 196608000$ bytes Capacity of zone 4: $16 \times 100 \times 280 \times 512 = 229376000$ bytes Sum = $131072000 + 163840000 + 196608000 + 229376000 = 720896000$

(b) A rotation rate of 7200 means there are 120 rotations/sec. In the 1 msec track-to-track seek time, 0.120 of the sectors are covered. In zone 1, the disk head will pass over $0.120 \times$

160 sectors in 1 msec, so, optimal track skew for zone 1 is 19.2 sectors. In zone 2, the disk head will pass over 0.120×200 sectors in 1 msec, so, optimal track skew for zone 2 is 24 sectors. In zone 3, the disk head will pass over 0.120×240 sectors in 1 msec, so, optimal track skew for zone 3 is 28.8 sectors. In zone 4, the disk head will pass over 0.120×280 sectors in 1 msec, so, optimal track skew for zone 3 is 33.6 sectors.

(c) The maximum data transfer rate will be when the cylinders in the outermost zone (zone 4) are being read/written. In that zone, in one second, 280 sectors are read 120 times. Thus the data rate is $280 \times 120 \times 512 = 17,203,200$ bytes/sec.

Source: Answer manual.

Math explanation. For each zone, capacity is heads $\times$ cylinders $\times$ sectors/cylinder $\times$ bytes/sector. Track skew comes from the fraction of a rotation spent seeking: $1 \text{ msec}/8.33 \text{ msec} \approx 0.12$. Multiply that fraction by sectors/track for each zone to get the needed sector skew.

Source: Prepared from textbook concepts.

Exercise 29

Question

A disk manufacturer has two 5.25-inch disks that each have 10,000 cylinders. The newer one has double the linear recording density of the older one. Which disk properties are better on the newer drive and which are the same? Are any worse on the newer one?

Answer

The drive capacity and transfer rates are doubled. The seek time and average rotational delay are the same. No properties are worse.

Source: Answer manual.

Exercise 30

Question

A computer manufacturer decides to redesign the partition table of a Pentium hard disk to provide more than four partitions. What are some consequences of this change?

Answer

One fairly obvious consequence is that no existing operating system will work because they all look there to see where the disk partitions are. Changing the format of the partition table will cause all the operating systems to fail. The only way to change the partition

table is to simultaneously change all the operating systems to use the new format.

Source: Answer manual.

Exercise 31

Question

Disk requests come in to the disk driver for cylinders 10, 22, 20, 2, 40, 6, and 38, in that order. A seek takes 6 msec per cylinder. How much seek time is needed for

- (a) First-come, first served.
- (b) Closest cylinder next.
- (c) Elevator algorithm (initially moving upward). In all cases, the arm is initially at cylinder 20.

Answer

- (a) $10 + 12 + 2 + 18 + 38 + 34 + 32 = 146$ cylinders = 876 msec.
- (b) $0 + 2 + 12 + 4 + 4 + 36 + 2 = 60$ cylinders = 360 msec.
- (c) $0 + 2 + 16 + 2 + 30 + 4 + 4 = 58$ cylinders = 348 msec.

Source: Answer manual.

Math explanation. Seek time is total cylinder movement multiplied by 6 msec/cylinder. For FCFS, the movements sum to 146 cylinders, so $146 \times 6 = 876$ msec. Closest-cylinder-next sums to 60 cylinders, giving 360 msec. Elevator sums to 58 cylinders, giving 348 msec.

Source: Prepared from textbook concepts.

Exercise 32

Question

A slight modification of the elevator algorithm for scheduling disk requests is to always scan in the same direction. In what respect is this modified algorithm better than the elevator algorithm?

Answer

In the worst case, a read/write request is not serviced for almost two full disk scans in the elevator algorithm, while it is at most one full disk scan in the modified algorithm.

Source: Answer manual.

Exercise 33

Question

A personal computer salesman visiting a university in South-West Amsterdam remarked during his sales pitch that his company had devoted substantial effort to making their version of UNIX very fast. As an example, he noted that their disk driver used the elevator algorithm and also queued multiple requests within a cylinder in sector order. A student, Harry Hacker, was impressed and bought one. He took it home and wrote a program to randomly read 10,000 blocks spread across the disk. To his amazement, the performance that he measured was identical to what would be expected from first-come, first-served. Was the salesman lying?

Answer

Not necessarily. A UNIX program that reads 10,000 blocks issues the requests one at a time, blocking after each one is issued until after it is completed. Thus the disk driver sees only one request at a time; it has no opportunity to do anything but process them in the order of arrival. Harry should have started up many processes at the same time to see if the elevator algorithm worked.

Manual numbering note: matched to answer-manual item 34 by content.

Source: Answer manual.

Exercise 34

Question

In the discussion of stable storage using nonvolatile RAM, the following point was glossed over. What happens if the stable write completes but a crash occurs before the operating system can write an invalid block number in the nonvolatile RAM? Does this race condition ruin the abstraction of stable storage? Explain your answer.

Answer

There is a race but it does not matter. Since the stable write itself has already completed, the fact that the nonvolatile RAM has not been updated just means that the recovery program will know which block was being written. It will read both copies. Finding them identical, it will change neither, which is the correct action. The effect of the crash just before the nonvolatile RAM was updated just means the recovery program will have to make two disk reads more than it should.

Manual numbering note: matched to answer-manual item 35 by content.

Source: Answer manual.

Exercise 35

Question

In the discussion on stable storage, it was shown that the disk can be recovered to a consistent state (a write either completes or does not take place at all) if a CPU crash occurs during a write. Does this property hold if the CPU crashes again during a recovery procedure. Explain your answer.

Answer

Yes the disk remains consistent even if the CPU crashes during a recovery procedure. Consider Fig. 5-31. There is no recovery involved in (a) or (e). Suppose that the CPU crashes during recovery in (b). If CPU crashes before the block from drive 2 has been completely copied to drive 1, the situation remains same as earlier. The subsequent recovery procedure will detect an ECC error in drive 1 and again copy the block from drive 2 to drive 1. If CPU crashes after the block from drive 2 has been copied to drive 1, the situation is same as that in case (e). Suppose that the CPU crashes during recovery in (c). If CPU crashes before the block from drive 1 has been completely copied to drive 2, the situation is same as that in case (d). The subsequent recovery procedure will detect an ECC error in drive 2 and copy the block from drive 1 to drive 2. If CPU crashes after the block from drive 1 has been copied to drive 2, the situation is same as that in case (e). Finally, suppose the CPU crashes during recovery in (d). If CPU crashes before the block from drive 1 has been completely copied to drive 2, the situation remains same as earlier. The subsequent recovery procedure will detect an ECC error in drive 2 and again copy the block from drive 1 to drive 2. If CPU crashes after the block from drive 1 has been copied to drive 2, the situation is same as that in case (e).
Manual numbering note: matched to answer-manual item 36 by content.

Source: Answer manual.

Exercise 36

Question

In the discussion on stable storage, a key assumption is that a CPU crash that corrupts a sector leads to an incorrect ECC. What problems might arise in the five crash-recovery scenarios shown in Figure 5-27 if this assumption does not hold?

Answer

Problems arise in scenarios shown in Figure 5-27 (b) and 5-27 (d), because they may look like scenario 5-27 (c), if the ECC of the corrupted block is correct. In this case, it is not possible to detect which disk contains the valid (old or new) lock, and a recovery is not possible.

Manual numbering note: matched to answer-manual item 37 by content.

Source: Answer manual.

Exercise 37

Question

The clock interrupt handler on a certain computer requires 2 msec (including process switching overhead) per clock tick. The clock runs at 60 Hz. What fraction of the CPU is devoted to the clock?

Answer

Two msec 60 times a second is 120 msec/sec, or 12% of the CPU

Manual numbering note: matched to answer-manual item 38 by content.

Source: Answer manual.

Math explanation. A 60-Hz clock ticks 60 times/sec. Each tick costs 2 msec, so per second the handler consumes $60 \times 2 = 120$ msec. The CPU fraction is $120/1000 = 0.12$, or 12%.

Source: Prepared from textbook concepts.

Exercise 38

Question

A computer uses a programmable clock in square-wave mode. If a 500 MHz crystal is used, what should be the value of the holding register to achieve a clock resolution of

- (a) a millisecond (a clock tick once every millisecond)?
- (b) 100 microseconds?

Answer

With these parameters,

(a) Using a 500 MHz crystal, the counter can be decremented every 2 nsec. So, for a tick every millisecond, the register should be $1000000/2 = 500,000$.

(b) To get a clock tick every 100 μ sec, holding register value should be 50,000.

Manual numbering note: matched to answer-manual item 39 by content.

Source: Answer manual.

Math explanation. A 500-MHz crystal has period $1/500,000,000 = 2$ nsec. For 1 msec, the count is $1,000,000/2 = 500,000$. For 100 microseconds, the count is $100,000/2 = 50,000$.

Source: Prepared from textbook concepts.

Exercise 39

Question

A system simulates multiple clocks by chaining all pending clock requests together as shown in Fig. 5-30. Suppose the current time is 5000 and there are pending clock requests for time 5008, 5012, 5015, 5029, and 5037. Show the values of Clock header, Current time, and Next signal at times 5000, 5005, and 5013. Suppose a new (pending) signal arrives at time 5017 for 5033. Show the values of Clock header, Current time and Next signal at time 5023.

Answer

At time 5000: Current time = 5000; Next Signal = 8; Header $\rightarrow 8 \rightarrow 4 \rightarrow 3 \rightarrow 14 \rightarrow 8$. At time 5005: Current time = 5005; Next Signal = 3; Header $\rightarrow 3 \rightarrow 4 \rightarrow 3 \rightarrow 14 \rightarrow 8$. At time 5013: Current time = 5013; Next Signal = 2; Header $2 \rightarrow 14 \rightarrow 8$. At time 5023: Current time = 5023; Next Signal = 6; Header $\rightarrow 6 \rightarrow 4 \rightarrow 5$.

Manual numbering note: matched to answer-manual item 40 by content.

Source: Answer manual.

Exercise 40

Question

Many versions of UNIX use an unsigned 32-bit integer to keep track of the time as the number of seconds since the origin of time. When will these systems wrap around (year and month)? Do you expect this to actually happen?

Answer

The number of seconds in a mean year is $365.25 \times 24 \times 3600$. This number is 31,557,600. The counter wraps around after 2^{32} seconds from 1 January 1970. The value of $2^{32} / 31,557,600$ is 136.1 years, so wrapping will happen at 2106.1, which is early February 2106. Of course, by then, all computers will be at least 64 bits, so it will not happen at all.

Manual numbering note: matched to answer-manual item 41 by content.

Source: Answer manual.

Math explanation. The counter has 2^{32} seconds. A mean year has $365.25 \times 24 \times 3600 = 31,557,600$ seconds. Dividing gives $2^{32} / 31,557,600 \approx 136.1$ years after January 1, 1970, which lands in early February 2106.

Source: Prepared from textbook concepts.

Exercise 41

Question

A bitmap terminal contains 1600 by 1200 pixels. To scroll a window, the CPU (or controller) must move all the lines of text upward by copying their bits from one part of the video RAM to another. If a particular window is 80 lines high by 80 characters wide (6400 characters, total), and a character's box is 8 pixels wide by 16 pixels high, how long does it take to scroll the whole window at a copying rate of 50 nsec per byte? If all lines are 80 characters long, what is the equivalent baud rate of the terminal? Putting a character on the screen takes 5 μ sec. How many lines per second can be displayed?

Answer

Scrolling the window requires copying 79 lines of 80 characters or 6320 characters. Copying 1 character (16 bytes) takes 800 nsec, so the whole window takes 5.056 msec. Writing 80 characters to the screen takes 400 nsec, so scrolling and displaying a new line take 5.456 msec. This gives about 183.2 lines/sec.

Manual numbering note: matched to answer-manual item 42 by content.

Source: Answer manual.

Math explanation. Scrolling moves 79 old lines because one new line enters at the bottom. That is $79 \times 80 = 6320$ characters. Each character cell is $8 \times 16 = 128$ bits = 16 bytes. At 50 nsec/byte, one character costs 800 nsec and the scroll copy costs 6320×800 nsec = 5.056 msec.

Source: Prepared from textbook concepts.

Exercise 42

Question

After receiving a DEL (SIGINT) character, the display driver discards all output currently queued for that display. Why?

Answer

Suppose that the user inadvertently asked the editor to print thousands of lines. Then he hits DEL to stop it. If the driver did not discard output, output might continue for several seconds after the DEL, which would make the user hit DEL again and again and get frustrated when nothing happened.

Manual numbering note: matched to answer-manual item 43 by content.

Source: Answer manual.

Exercise 43

Question

A user at a terminal issues a command to an editor to delete the word on line 5 occupying character positions 7 through and including 12. Assuming the cursor is not on line 5 when the command is given, what ANSI escape sequence should the editor emit to delete the word?

Answer

It should move the cursor to line 5 position 7 and then delete 6 characters. The sequence is `ESC [ 5 ; 7 H ESC [ 6 P`

Manual numbering note: matched to answer-manual item 44 by content.

Source: Answer manual.

Exercise 44

Question

The designers of a computer system expected that the mouse could be moved at a maximum rate of 20 cm/sec. If a mickey is 0.1 mm and each mouse message is 3 bytes, what is the maximum data rate of the mouse assuming that each mickey is reported separately?

Answer

The maximum rate the mouse can move is 200 mm/sec, which is 2000 mickeys/sec. If each report is 3 byte, the output rate is 6000 bytes/sec.

Manual numbering note: matched to answer-manual item 45 by content.

Source: Answer manual.

Math explanation. A speed of 20 cm/sec is 200 mm/sec. One mickey is 0.1 mm, so the mouse reports $200/0.1 = 2000$ mickeys/sec. At 3 bytes per report, the data rate is $2000 \times 3 = 6000$ bytes/sec.

Source: Prepared from textbook concepts.

Exercise 45

Question

The primary additive colors are red, green, and blue, which means that any color can be constructed from a linear superposition of these colors. Is it possible that someone could have a color photograph that cannot be represented using full 24-bit color?

Answer

With a 24-bit color system, only 2^{24} colors can be represented. This is not all of them. For example, suppose that a photographer takes pictures of 300 cans of pure blue paint, each with a slightly different amount of pigment. The first might be represented by the (R, G, B) value (0, 0, 1). The next one might be represented by (0, 0, 2), and so forth. Since the B coordinate is only 8 bits, there is no way to represent 300 different values of pure blue. Some of the photographs will have to be rendered as the wrong color. Another example is the color (120.24, 150.47, 135.89). It cannot be represented, only approximated by (120, 150, 136).

Manual numbering note: matched to answer-manual item 46 by content.

Source: Answer manual.

Exercise 46

Question

One way to place a character on a bitmapped screen is to use BitBlt from a font table. Assume that a particular font uses characters that are 16×24 pixels in true RGB color.

- (a) How much font table space does each character take?
- (b) If copying a byte takes 100 nsec, including overhead, what is the output rate to the screen in characters/sec?

Answer

(a) Each pixel takes 3 bytes in RGB, so the table space is $16 \times 24 \times 3$ bytes, which is 1152 bytes.

(b) At 100 nsec per byte, each character takes 115.2 μ sec. This gives an output rate of about 8681 chars/sec.

Manual numbering note: matched to answer-manual item 47 by content.

Source: Answer manual.

Math explanation. A character bitmap has $16 \times 24 = 384$ pixels. True RGB color uses 3 bytes/pixel, so one character needs $384 \times 3 = 1152$ bytes. At 100 nsec per byte, output time is $1152 \times 100 = 115,200$ nsec = 115.2 microseconds, giving $1/115.2 \mu s \approx 8681$ chars/sec.

Source: Prepared from textbook concepts.

Exercise 47

Question

Assuming that it takes 2 nsec to copy a byte, how much time does it take to completely rewrite the screen of an 80 character $\times$ 25 line text mode memory-mapped screen? What about a 1024 $\times$ 768 pixel graphics screen with 24-bit color?

Answer

Rewriting the text screen requires copying 2000 bytes, which can be done in 4 μ seconds. Rewriting the graphics screen requires copying $1024 \times 768 \times 3 = 2,359,296$ bytes, or about 4.72 msec.

Manual numbering note: matched to answer-manual item 48 by content.

Source: Answer manual.

Math explanation. Text mode has $80 \times 25 = 2000$ bytes, so at 2 nsec/byte it takes 4000 nsec = 4 microseconds. A 1024 by 768 screen with 24-bit color has $1024 \times 768 \times 3 = 2,359,296$ bytes, which takes $2,359,296 \times 2 \text{ nsec} = 4.72 \text{ msec}$.

Source: Prepared from textbook concepts.

Exercise 48

Question

In Fig. 5-36 there is a call to RegisterClass. In the corresponding X Window code, in Fig. 5-34, there is no such call or anything like it. Why not?

Answer

In Windows, the OS calls the handler procedures itself. In X Windows, nothing like this happens. X just gets a message and processes it internally.

Manual numbering note: matched to answer-manual item 49 by content.

Source: Answer manual.

Exercise 49

Question

In the text we gave an example of how to draw a rectangle on the screen using the Windows GDI: `Rectangle(hdc, xleft, ytop, xright, ybottom)`; Is there any real need for the first parameter (hdc), and if so, what? After all, the coordinates of the rectangle are

explicitly specified as parameters.

Answer

The first parameter is essential. First of all, the coordinates are relative to some window, so `hdc` is needed to specify the window and thus the origin. Second, the rectangle will be clipped if it falls outside the window, so the window coordinates are needed. Third, the color and other properties of the rectangle are taken from the context specified by `hdc`. It is quite essential.

Manual numbering note: matched to answer-manual item 50 by content.

Source: Answer manual.

Exercise 50

Question

A thin-client terminal is used to display a Web page containing an animated cartoon of size $400 \text{ pixels} \times 160 \text{ pixels}$ running at 10 frames/sec. What fraction of a 100-Mbps Fast Ethernet is consumed by displaying the cartoon?

Answer

The display size is $400 \times 160 \times 3$ bytes, which is 192,000 bytes. At 10 fps this is 1,920,000 bytes/sec or 15,360,000 bits/sec. This consumes 15% of the Fast Ethernet.

Manual numbering note: matched to answer-manual item 51 by content.

Source: Answer manual.

Math explanation. Each frame has 400×160 pixels. At 3 bytes/pixel, that is 192,000 bytes/frame. At 10 frames/sec, the stream is 1,920,000 bytes/sec = 15,360,000 bits/sec. Dividing by 100 Mbit/sec gives 0.1536, or about 15%.

Source: Prepared from textbook concepts.

Exercise 51

Question

It has been observed that a thin-client system works well with a 1-Mbps network in a test. Are any problems likely in a multiuser situation? (Hint: Consider a large number of users watching a scheduled TV show and the same number of users browsing the World Wide Web.)

Answer

The bandwidth on a network segment is shared, so 100 users requesting different data simultaneously on a 1-Mbps network will each see a 10-Kbps effective speed. With a shared network, a TV program can be multicast, so the video packets are only broadcast once, no matter how many users there are and it should work well. With 100 users browsing the Web, each user will get 1/100 of the bandwidth, so performance may degrade very quickly. *Manual numbering note: matched to answer-manual item 52 by content.*

Source: Answer manual.

Exercise 52

Question

Describe two advantages and two disadvantages of thin client computing?

Answer

Advantages of thin clients include low cost and no need for complex management for the clients. Disadvantages include (potentially) lower performance due to network latency and (potential) loss of privacy because the client's data/information is shared with the server.

Manual numbering note: matched to answer-manual item 53 by content.

Source: Answer manual.

Exercise 53

Question

If a CPU's maximum voltage, V , is cut to V/n , its power consumption drops to $1/n^2$ of its original value and its clock speed drops to $1/n$ of its original value. Suppose that a user is typing at 1 char/sec, but the CPU time required to process each character is 100 msec. What is the optimal value of n and what is the corresponding energy saving in percent compared to not cutting the voltage? Assume that an idle CPU consumes no energy at all.

Answer

If $n = 10$, the CPU can still get its work done on time, but the energy used drops appreciably. If the energy consumed in 1 sec at full speed is E , then running at full speed for 100 msec then going idle for 900 msec uses $E/10$. Running at $1/10$ speed for a whole second uses $E/100$, a saving of $9E/100$. The percent savings by cutting the voltage is 90%.

Manual numbering note: matched to answer-manual item 54 by content.

Source: Answer manual.

Math explanation. At full speed, each character needs 100 msec of CPU time, and characters arrive once/sec. Slowing the clock by $n = 10$ stretches 100 msec of work to 1 sec, exactly filling the available time. Power drops to $1/n^2 = 1/100$. Compared with running full speed for 0.1 sec, which uses $E/10$, slow running for the full second uses $E/100$, saving $E/10 - E/100 = 9E/100$, i.e., 90% of the original active energy.

Source: Prepared from textbook concepts.

Exercise 54

Question

A notebook computer is set up to take maximum advantage of power saving features including shutting down the display and the hard disk after periods of inactivity. A user sometimes runs UNIX programs in text mode, and at other times uses the X Window System. She is surprised to find that battery life is significantly better when she uses text-only programs. Why?

Answer

The windowing system uses much more memory for its display and uses virtual memory more than the text mode. This makes it less likely that the hard disk will be inactive for a period long enough to cause it to be automatically powered down.

Manual numbering note: matched to answer-manual item 55 by content.

Source: Answer manual.

Exercise 55

Question

Write a program that simulates stable storage. Use two large fixed-length files on your disk to simulate the two disks.

Answer

Use two fixed-length files as the two copies of each stable-storage block. A stable write writes copy 1, verifies it, writes copy 2, verifies it, and records enough metadata to recover after a simulated crash.

```
BLOCK = 4096

def read_block(path, blockno):
    with open(path, "rb") as f:
        f.seek(blockno * BLOCK)
```

```

        return f.read(BLOCK)

def write_block(path, blockno, data):
    data = data.ljust(BLOCK, b"\0")[:BLOCK]
    with open(path, "r+b") as f:
        f.seek(blockno * BLOCK)
        f.write(data)
        f.flush()
        # In a real test, also use os.fsync(f.fileno()).

def recover(d1, d2, blockno):
    a = read_block(d1, blockno)
    b = read_block(d2, blockno)
    if a != b:
        # With checksums, choose the valid copy. Without them, report
        ambiguity.
        raise RuntimeError("copies differ; checksum metadata needed")

def stable_write(d1, d2, blockno, data):
    write_block(d1, blockno, data)
    write_block(d2, blockno, data)

```

Expected behavior: after a normal stable write, both simulated disks contain the same block. To test crashes safely, inject exceptions between the writes and run recovery on small temporary files, not important data. Safety warning: this is a user-space simulation; real stable storage also needs checksums/ECC assumptions, fsync or equivalent flushes, and careful handling of torn writes.

Source: Independently prepared technical answer.

Exercise 56

Question

Write a program to implement the three disk-arm scheduling algorithms. Write a driver program that generates a sequence of cylinder numbers (0–999) at random, runs the three algorithms for this sequence and prints out the total distance (number of cylinders) the arm needs to traverse in the three algorithms.

Answer

Generate one request sequence and feed the same sequence to all three algorithms. The output should be total arm movement in cylinders.

```

def fcfs(reqs, start):
    pos, total = start, 0
    for r in reqs:
        total += abs(pos - r)
        pos = r
    return total

def sstf(reqs, start):
    pending, pos, total = reqs[:], start, 0
    while pending:
        r = min(pending, key=lambda x: abs(x - pos))
        total += abs(pos - r)
        pos = r
        pending.remove(r)
    return total

def elevator_up(reqs, start):
    up = sorted(r for r in reqs if r >= start)
    down = sorted((r for r in reqs if r < start), reverse=True)
    return fcfs(up + down, start)

```

Expected behavior: FCFS depends heavily on arrival order; closest-cylinder-next usually reduces movement but can starve far requests; elevator gives more regular service. Safer testing method: first use the textbook sequence to verify the totals, then use random sequences from 0 to 999. No hardware disk scheduling is performed by this program.

Source: Independently prepared technical answer.

Exercise 57

Question

Write a program to implement multiple timers using a single clock. Input for this program consists of a sequence of four types of commands (S <int>, T, E <int>, P): S <int> sets the current time to <int>; T is a clock tick; and E <int> schedules a signal to occur at time <int>; P prints out the values of Current time, Next signal, and Clock header. Your program should also print out a statement whenever it is time to raise a signal.

Answer

Store pending timer events as deltas from the previous event, like the linked list in the chapter. A clock tick decrements only the first delta. When it reaches zero, raise that

signal and continue with the next event.

```
current = 0
events = [] # absolute times, kept sorted

def schedule(t):
    events.append(t)
    events.sort()

def tick():
    global current
    current += 1
    while events and events[0] <= current:
        print("signal", events.pop(0))

def print_state():
    next_signal = None if not events else events[0] - current
    deltas = []
    prev = current
    for t in events:
        deltas.append(t - prev)
        prev = t
    print("Current time =", current)
    print("Next signal =", next_signal)
    print("Clock header =", deltas)
```

Expected behavior: command `S n` sets the current time, `T` advances it by one tick and prints any due signals, `E n` schedules a future signal, and `P` prints the current list state. Safety note: reject events scheduled in the past or define that they fire immediately; otherwise the delta list can become negative.

Source: Independently prepared technical answer.